
Tribhuvan University
Faculty of Humanities & Social Sciences
OFFICE OF THE DEAN

C Programming

BCA — Semester 2 — 2018 — Answer Sheet

Subject Code: CACS151

Group A

Attempt ALL the questions. [10 × 1 = 10]

-
1. 1. Which of the following doesn't require an & for the input in scanf()?
a) char name[10] b) int name[10] c) float name[10] d) double name[10]
 2. What is the memory size of float data type in C?
a) 4 Bytes b) 8 Bytes c) Depends on system/compiler d) Cannot be determined
 3. What will be the output of the following C code?
int x=3,y; y=(++x)+(x++); printf("%d",y);
a) 6 b) 8 c) 7 d) 9
 4. Which keyword is used to come out of a loop only for that iteration?
a) break b) continue c) return d) void
 5. Bitwise operators can operate upon?
a) Double and chars b) floats and doubles c) int and floats d) int and chars
 6. In C, if you pass an array as an argument to a function, what actually gets passed?
a) Value of elements b) First element c) Address of first element d) Address of last element
 7. Which operator is used to access members of structure using structure variable?
a) Address operator (&) b) Dot operator (.) c) Pointer operator (*) d) Arrow operator (->)
 8. Which function is used to record input from file?
a) ftell() b) fwrite() c) fprintf() d) fread()
 9. Which of the following is a keyword used for a storage class?
a) printf b) goto c) external d) break
 10. What will be the size of following union declaration?
union test { int x; char y; float z; }
a) 8 bytes b) 13 bytes c) 1 byte d) 4 bytes

[10 × 1 = 10]

Answer: Answers:

1) a) char name[10] — Array names are pointers to the first element, so they don't need &. For int/float/double arrays, each element needs & when reading individual elements.

2) a) 4 Bytes — In standard C, float typically occupies 4 bytes (32 bits) with precision of about 6-7 decimal digits.

3) b) 8 — ++x (pre-increment) makes x=4, then x++ (post-increment) uses x=4 and makes x=5. So y = 4 + 4 = 8.

4) b) continue — continue skips the current iteration and moves to the next iteration. break exits the loop entirely.

5) d) int and chars — Bitwise operators (&, |, ^, ~, <<, >>) work on integer types (int, char, short, long). They cannot operate on float or double.

6) c) The address of first element in an array — Arrays decay to pointers when passed to functions. The

function receives the base address, not the values.

7) b) Dot operator (.) — The dot operator accesses structure members through a structure variable. Arrow operator (->) is used with structure pointers.

8) d) fread() — fread() reads binary data from a file. fwrite() writes binary data. fprintf() writes formatted text. ftell() returns file position.

9) c) external — extern is a storage class keyword (though misspelled as "external" in options). auto, static, register, and extern are C storage classes.

10) d) 4 bytes — Union size equals the size of its largest member. Here float z = 4 bytes (assuming 32-bit system), so union size = 4 bytes.

Group B

Attempt any SIX questions. $[6 \times 5 = 30]$

11. What is software process model? Differentiate between cohesion and coupling in programming. [1+4]

[5]

Answer: Software Process Model:

A software process model is a structured set of activities required to develop a software system. It defines the order of activities, their inputs/outputs, and the workflow. Common models include Waterfall, Agile, Spiral, and V-Model.

Cohesion vs Coupling:

Cohesion Measures how closely related elements within a module are. Higher cohesion is better. **Coupling** Measures the degree of interdependence between modules. Lower coupling is better. **Types:** Functional, Sequential, Communicational, Procedural, Temporal, Logical, Coincidental. **Types:** Content, Common, External, Control, Stamp, Data. **Goal:** Each module should do one thing well. **Goal:** Modules should be as independent as possible.

Example: A module that calculates salary (functional cohesion) is better than one that does unrelated tasks (coincidental cohesion). Two modules sharing only necessary data (data coupling) is better than one module controlling another's logic (control coupling).

12. Define keyword and identifiers. Explain rules for defining valid identifiers. [2+3]

[5]

Answer: Keywords:

Keywords are reserved words in C that have predefined meanings and cannot be used as identifiers. They are the building blocks of C syntax. Examples: int, float, if, else, for, while, return, struct, void, static.

Identifiers:

Identifiers are names given to variables, functions, arrays, structures, and other user-defined elements. They allow programmers to reference data and operations symbolically.

Rules for Valid Identifiers:

1. Must begin with a letter (a-z, A-Z) or underscore (_).
2. Can contain letters, digits (0-9), and underscores.
3. Cannot contain special characters (@, #, \$, %, space, etc.).
4. Cannot be a C keyword (if, int, for, etc.).
5. C is case-sensitive: "name" and "Name" are different identifiers.
6. Maximum length depends on compiler, but first 31 characters are significant in standard C.

Valid: _count, totalSum, student_name, value2

Invalid: 2value (starts with digit), student name (contains space), int (keyword), total\$sum (special char)

13. List the operators used in C on the basis of utility. Explain the concept of bitwise operator. [2+3]

[5]

Answer: Operators in C by Utility:

Category Operators Arithmetic +, -, *, /, % Relational <, >, <=, >=, ==, != Logical &&, ||, ! Bitwise &, |, ^, ~, <<, >> Assignment =, +=, -=, *=, /=, %= Increment/Decrement ++, -- Conditional?: Special sizeof, &, *, , (comma)

Bitwise Operators:

Bitwise operators perform operations on individual bits of integer operands.

& (AND): Sets bit to 1 if both bits are 1.

$5 \& 3 = 101 \& 011 = 001 = 1$

| (OR): Sets bit to 1 if at least one bit is 1.

$5 | 3 = 101 | 011 = 111 = 7$

^ (XOR): Sets bit to 1 if bits are different.

$5 \wedge 3 = 101 \wedge 011 = 110 = 6$

~ (NOT): Inverts all bits (one's complement).

$\sim 5 = \sim 00000101 = 11111010 = -6$ (in 2's complement)

<< (Left Shift): Shifts bits left, filling with 0.

$5 \ll 1 = 1010 = 10$ (equivalent to $\times 2$)

>> (Right Shift): Shifts bits right.

$5 \gg 1 = 0010 = 2$ (equivalent to $\div 2$)

14. Differentiate between while loop and do while loop. Write a C program to find input number is prime or composite. [2+3]

[5]

Answer: Difference between while and do-while:

while loop **do-while loop** Condition checked before entering loop Condition checked after executing loop body
May not execute at all (0 iterations) Executes at least once (minimum 1 iteration) Syntax: while(condition) {
} Syntax: do { } while(condition); Entry-controlled loop Exit-controlled loop

C Program to check Prime or Composite:

```
#include<stdio.h> int main() { int n, i, flag = 0; printf("Enter a number: "); scanf("%d", &n); if(n <= 1) { printf("%d  
is neither prime nor composite\n", n); } else { for(i = 2; i <= n/2; i++) { if(n % i == 0) { flag = 1; break; } } if(flag ==  
0) printf("%d is a prime number\n", n); else printf("%d is a composite number\n", n); } return 0; }
```

Explanation: The program checks divisibility from 2 to n/2. If any divisor is found, the number is composite. Otherwise, it's prime. Numbers ≤ 1 are neither prime nor composite.

15. What is DMA? Write a program to find the largest and smallest number in a list of N numbers using DMA. [1+4]

[5]

Answer: DMA (Dynamic Memory Allocation):

DMA is the process of allocating memory during program execution (runtime) rather than at compile time. It allows programs to request memory as needed and release it when no longer required. C provides functions malloc(), calloc(), realloc(), and free() for DMA, declared in <stdlib.h>.

Program using DMA to find largest and smallest:

```
#include<stdio.h> #include<stdlib.h> int main() { int n, i, *arr, largest, smallest; printf("Enter number of  
elements: "); scanf("%d", &n); // Dynamic memory allocation arr = (int*)malloc(n * sizeof(int)); if(arr == NULL) {  
printf("Memory allocation failed!\n"); return 1; } printf("Enter %d numbers:\n", n); for(i = 0; i < n; i++) {  
scanf("%d", &arr[i]); } largest = smallest = arr[0]; for(i = 1; i < n; i++) { if(arr[i] > largest) largest = arr[i]; if(arr[i] <  
smallest) smallest = arr[i]; } printf("Largest = %d\n", largest); printf("Smallest = %d\n", smallest); free(arr); //  
Deallocate memory return 0; }
```

Explanation: malloc() allocates memory for n integers at runtime. The user can enter any number of elements. After processing, free() releases the memory to prevent memory leaks.

16. What is difference between binary file and text file? Write a C program to write some text "Welcome to BCA program" in a file test.txt. [2+3]

[5]

Answer: Text File vs Binary File:

Text File	Binary File
Stores data as ASCII characters	Stores data in binary format (0s and 1s)
Human-readable	Not human-readable
Uses functions: fprintf(), fscanf(), fgetc(), fputc()	Uses functions: fwrite(), fread()
Newline character converted to system-specific format	No translation, data stored as-is
Slower for numerical data	Faster and more compact for numerical data
Example: .txt, .csv, .html	Example: .exe, .bin, .dat, .jpg

C Program to write text to file:

```
#include<stdio.h> int main() { FILE *fp; fp = fopen("test.txt", "w"); if(fp == NULL) { printf("Error opening file!\n");  
return 1; } fprintf(fp, "Welcome to BCA program"); fclose(fp); printf("Text written successfully!\n"); return 0; }
```

Explanation: fopen() opens test.txt in write mode ("w"). fprintf() writes the string to the file. fclose() closes the file, flushing the buffer to disk. If the file doesn't exist, it's created; if it exists, its contents are overwritten.

17. Explain any four graphics functions in C. Write a program to draw two concentric circles with center (50, 50) and radii 75 and 125. [2+3]

[5]

Answer: Graphics Functions in C (using graphics.h):

1. initgraph(): Initializes the graphics system.

initgraph(&gd, &gm, "");

2. setcolor(): Sets the current drawing color.

setcolor(RED);

3. circle(): Draws a circle with given center and radius.

circle(x, y, radius);

4. line(): Draws a line between two points.

line(x1, y1, x2, y2);

Other functions: rectangle(), bar(), putpixel(), outtextxy(), closegraph()

Program to draw concentric circles:

```
#include<graphics.h> #include<conio.h> int main() { int gd = DETECT, gm; initgraph(&gd, &gm, ""); // Draw first circle with radius 75 setcolor(RED); circle(50, 50, 75); // Draw second circle with radius 125 setcolor(BLUE); circle(50, 50, 125); getch(); closegraph(); return 0; }
```

Explanation: Both circles share the same center (50, 50) but have different radii (75 and 125), creating concentric circles. The graphics system must be initialized with initgraph() and closed with closegraph(). getch() waits for a key press before closing.

Group C

Attempt any TWO questions. [2 × 10 = 20]

18. What is one dimensional array? How it is initialized? Write a C program to find the sum of two matrix of order m×n. [1+1+8]

[10]

Answer: One Dimensional Array:

A one-dimensional array is a linear collection of elements of the same data type stored in contiguous memory locations. It is accessed using a single index. Declaration: **type arrayName[size];**

Initialization:

int arr[5] = {10, 20, 30, 40, 50}; int arr[] = {10, 20, 30}; // Size determined automatically int arr[5] = {10, 20}; // Remaining elements set to 0

Program for Matrix Addition:

```
#include<stdio.h> int main() { int a[10][10], b[10][10], sum[10][10]; int m, n, i, j; printf("Enter rows and columns: "); scanf("%d%d", &m, &n); printf("Enter elements of first matrix:\n"); for(i = 0; i < m; i++) for(j = 0; j < n; j++) scanf("%d", &a[i][j]); printf("Enter elements of second matrix:\n"); for(i = 0; i < m; i++) for(j = 0; j < n; j++) scanf("%d", &b[i][j]); // Matrix addition for(i = 0; i < m; i++) for(j = 0; j < n; j++) sum[i][j] = a[i][j] + b[i][j]; printf("Sum of matrices:\n"); for(i = 0; i < m; i++) { for(j = 0; j < n; j++) printf("%d ", sum[i][j]); printf("\n"); } return 0; }
```

Explanation: Two matrices are read element by element. Corresponding elements are added and stored in the sum matrix. The result is displayed in matrix format.

19. Define structure and union? Write a C program using structure that reads the records of 35 students with members roll, name, address and marks and display the record of students who have obtained greater than 250 marks. [2+8]

[10]

Answer: Structure:

A structure is a user-defined data type that groups variables of different data types under a single name. Each variable in a structure is called a member. Structures are declared using the **struct** keyword. The total memory allocated is the sum of all members' sizes.

Union:

A union is similar to a structure but all members share the same memory location. Only one member can hold a value at a time. The size of a union equals the size of its largest member. Unions are declared using the **union** keyword.

Difference:

Structure	Union
All members have separate memory	All members share same memory
Size = sum of all members	Size = size of largest member
All members can hold values simultaneously	Only one member active at a time

Program using Structure:

```
#include<stdio.h> #include<string.h> struct Student { int roll; char name[50]; char address[100]; int marks; };
int main() { struct Student s[35]; int i, count = 0; printf("Enter records of 35 students:\n"); for(i = 0; i < 35; i++) {
printf("\nStudent %d:\n", i+1); printf("Roll: "); scanf("%d", &s[i].roll); printf("Name: "); scanf("%[^\n]", s[i].name);
printf("Address: "); scanf("%[^\n]", s[i].address); printf("Marks: "); scanf("%d", &s[i].marks); } printf("\nStudents
with marks > 250:\n"); for(i = 0; i < 35; i++) { if(s[i].marks > 250) { printf("Roll: %d, Name: %s, Address: %s,
Marks: %d\n", s[i].roll, s[i].name, s[i].address, s[i].marks); count++; } } if(count == 0) printf("No student scored
more than 250 marks.\n"); return 0; }
```

Explanation: A structure Student is defined with four members. An array of 35 structures stores student records. The program iterates through all records and displays only those with marks greater than 250.

20. What is function? List its advantages. Explain the concept of function call by value and function call by reference with example. [1+2+7]

[10]

Answer: Function:

A function is a self-contained block of code that performs a specific task. It is defined once and can be called multiple times from different parts of a program. Functions help in modular programming, code reuse, and easier debugging.

Advantages of Functions:

- 1. Modularity:** Breaks complex problems into smaller, manageable modules.
- 2. Code Reusability:** Write once, use multiple times.
- 3. Easier Debugging:** Errors can be isolated to specific functions.
- 4. Reduced Code Size:** Avoids repetition of code.
- 5. Team Collaboration:** Different programmers can work on different functions.

Call by Value:

A copy of the actual argument is passed to the function. Changes made to parameters inside the function do not affect the original variables.

Call by Reference:

The address (memory location) of the actual argument is passed. Changes made to parameters inside the function directly affect the original variables.

Example:

```
#include<stdio.h> // Call by value void swapByValue(int a, int b) { int temp = a; a = b; b = temp; printf("Inside swapByValue: a=%d, b=%d\n", a, b); } // Call by reference void swapByReference(int *a, int *b) { int temp = *a; *a = *b; *b = temp; printf("Inside swapByReference: a=%d, b=%d\n", *a, *b); } int main() { int x = 10, y = 20; printf("Original: x=%d, y=%d\n", x, y); swapByValue(x, y); printf("After call by value: x=%d, y=%d\n", x, y); // x and y remain unchanged swapByReference(&x, &y); printf("After call by reference: x=%d, y=%d\n", x, y); // x and y are swapped return 0; }
```

Output:

Original: x=10, y=20

Inside swapByValue: a=20, b=10

After call by value: x=10, y=20

Inside swapByReference: a=20, b=10

After call by reference: x=20, y=10

Conclusion: Call by value is safer (original data protected) but cannot modify caller's variables. Call by reference is more efficient for large data and allows modification of original variables.

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.